

```
// =====
// JSCAG Food Solutions - Commissary Inventory System
// Production Version: Supabase + Excel Export + Auth + Customizable
//
// ENV VARS NEEDED (set in Vercel):
//   VITE_SUPABASE_URL=https://xxxx.supabase.co
//   VITE_SUPABASE_ANON_KEY=eyJ...
//
// This file is the complete src/App.jsx
// Deploy instructions in the Deployment Guide PDF.
// =====

import { useState, useEffect, useMemo, createContext, useContext } from "react";

// — SUPABASE CLIENT —————
// In production, install: npm install @supabase/supabase-js
// and uncomment below:
//
// import { createClient } from '@supabase/supabase-js';
// const supabase = createClient(
//   import.meta.env.VITE_SUPABASE_URL,
//   import.meta.env.VITE_SUPABASE_ANON_KEY
// );
//
// For this demo, we use in-memory data (same as before).
// The Supabase calls are shown as comments next to each function.

// — EXCEL EXPORT HELPER —————
// In production: npm install xlsx
// import * as XLSX from 'xlsx';
function exportToExcel(data, sheetName, filename) {
  // Production version:
  // const ws = XLSX.utils.json_to_sheet(data);
  // const wb = XLSX.utils.book_new();
  // XLSX.utils.book_append_sheet(wb, ws, sheetName);
  // XLSX.writeFile(wb, filename + '.xlsx');

  // Demo version - exports as CSV:
  if (!data || data.length === 0) return;
  const keys = Object.keys(data[0]);
  const csv = [keys.join(","), ...data.map(r => keys.map(k => `"${r[k] ?? ""}"`).
  const blob = new Blob([csv], { type: "text/csv" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a"); a.href = url; a.download = filename + ".

```

```

    URL.revokeObjectURL(url);
}

// — SEED DATA —————
const TODAY = new Date().toISOString().split("T")[0];
const YESTERDAY = new Date(Date.now() - 86400000).toISOString().split("T")[0];

const INITIAL_SETTINGS = {
  companyName: "JSCAG Food Solutions",
  tagline: "Pioneer ng Modern Flavored Crispy Chicken Isaw (since 2019)",
  primaryColor: "#2563eb",
  currency: "₱",
  workingDaysPerMonth: 30,
  alertEmail: "bryan@jscag.com",
  lowStockAlerts: true,
  rejectRateThreshold: 5, // % - alert if above this
  timezone: "Asia/Manila",
};

const SEED_CATEGORIES = [
  { id: 1, name: "Kitchen", color: "#e74c3c", active: true, sort_order: 1 },
  { id: 2, name: "Packaging", color: "#3498db", active: true, sort_order: 2 },
  { id: 3, name: "Cleaning / Sanitation", color: "#2ecc71", active: true, sort_or
  { id: 4, name: "E-commerce", color: "#9b59b6", active: true, sort_order: 4 },
  { id: 5, name: "Office", color: "#f39c12", active: true, sort_order: 5 },
  { id: 6, name: "Others", color: "#95a5a6", active: true, sort_order: 6 },
];

const SEED_ITEMS = [
  { id: 1, name: "Chicken Isaw", categoryId: 1, unit: "kg", purchaseUnit: "kg", p
  { id: 2, name: "All-Purpose Flour", categoryId: 1, unit: "g", purchaseUnit: "1
  { id: 3, name: "Cornstarch", categoryId: 1, unit: "g", purchaseUnit: "1 kg", pu
  { id: 4, name: "Seasoning Mix (BBQ)", categoryId: 1, unit: "g", purchaseUnit: "
  { id: 5, name: "Cooking Oil", categoryId: 1, unit: "ml", purchaseUnit: "1 liter
  { id: 6, name: "Plastic Packaging (50g)", categoryId: 2, unit: "pcs", purchaseU
  { id: 7, name: "Sticker Label", categoryId: 2, unit: "pcs", purchaseUnit: "roll
  { id: 8, name: "Dishwashing Liquid", categoryId: 3, unit: "ml", purchaseUnit: "
  { id: 9, name: "Alcohol 70%", categoryId: 3, unit: "ml", purchaseUnit: "1 liter
];

const SEED_PRODUCTS = [
  { id: 1, name: "J's Crispy Isaw - Classic", sku: "JCI-CL-50", targetPackSize: "
  { id: 2, name: "J's Crispy Isaw - BBQ", sku: "JCI-BBQ-50", targetPackSize: "50g
  { id: 3, name: "J's Crispy Isaw - Salted Egg", sku: "JCI-SE-50", targetPackSize
  { id: 4, name: "J's Crispy Isaw - Plain w/ Suka", sku: "JCI-PL-50", targetPackS
  { id: 5, name: "J's Lettuce Crisps", sku: "JLC-50", targetPackSize: "50g", sell
];

```

```
const SEED_INVENTORY = [
  { id: 1, date: YESTERDAY, itemId: 1, categoryId: 1, beginning: 15, ending: 8, u
  { id: 2, date: YESTERDAY, itemId: 2, categoryId: 1, beginning: 5000, ending: 28
  { id: 3, date: YESTERDAY, itemId: 3, categoryId: 1, beginning: 3000, ending: 15
  { id: 4, date: YESTERDAY, itemId: 4, categoryId: 1, beginning: 600, ending: 200
  { id: 5, date: YESTERDAY, itemId: 5, categoryId: 1, beginning: 5000, ending: 20
  { id: 6, date: YESTERDAY, itemId: 6, categoryId: 2, beginning: 2000, ending: 80
  { id: 7, date: YESTERDAY, itemId: 7, categoryId: 2, beginning: 1000, ending: 30
  { id: 8, date: TODAY, itemId: 1, categoryId: 1, beginning: 8, ending: 3.5, unit
  { id: 9, date: TODAY, itemId: 2, categoryId: 1, beginning: 2800, ending: 1200,
  { id: 10, date: TODAY, itemId: 3, categoryId: 1, beginning: 1500, ending: 700,
  { id: 11, date: TODAY, itemId: 4, categoryId: 1, beginning: 200, ending: 0, uni
  { id: 12, date: TODAY, itemId: 5, categoryId: 1, beginning: 2000, ending: 800,
  { id: 13, date: TODAY, itemId: 6, categoryId: 2, beginning: 800, ending: 0, uni
  { id: 14, date: TODAY, itemId: 7, categoryId: 2, beginning: 300, ending: 0, uni
];
```

```
const SEED_PRODUCTION = [
  { id: 1, date: YESTERDAY, productId: 1, batchNo: "B-001", goodPacks: 820, rejec
  { id: 2, date: YESTERDAY, productId: 2, batchNo: "B-002", goodPacks: 350, rejec
  { id: 3, date: TODAY, productId: 1, batchNo: "B-003", goodPacks: 780, rejects:
  { id: 4, date: TODAY, productId: 2, batchNo: "B-004", goodPacks: 220, rejects:
];
```

```
const SEED_PAYROLL = [
  { id: 1, date: YESTERDAY, employee: "Jose Cruz", role: "Production Staff", dail
  { id: 2, date: YESTERDAY, employee: "Maria Santos", role: "Inventory Auditor",
  { id: 3, date: YESTERDAY, employee: "Ana Reyes", role: "Packing Staff", dailyRa
  { id: 4, date: TODAY, employee: "Jose Cruz", role: "Production Staff", dailyRat
  { id: 5, date: TODAY, employee: "Maria Santos", role: "Inventory Auditor", dail
  { id: 6, date: TODAY, employee: "Ana Reyes", role: "Packing Staff", dailyRate:
];
```

```
const SEED_OVERHEAD = [
  { id: 1, date: YESTERDAY, costType: "Electricity", amount: 500, allocationType:
  { id: 2, date: YESTERDAY, costType: "Gas / LPG", amount: 350, allocationType: "
  { id: 3, date: YESTERDAY, costType: "Rent", amount: 1000, allocationType: "dail
  { id: 4, date: TODAY, costType: "Electricity", amount: 500, allocationType: "da
  { id: 5, date: TODAY, costType: "Gas / LPG", amount: 350, allocationType: "dail
  { id: 6, date: TODAY, costType: "Rent", amount: 1000, allocationType: "daily",
  { id: 7, date: TODAY, costType: "Water", amount: 100, allocationType: "daily",
  { id: 8, date: TODAY, costType: "Transportation", amount: 200, allocationType:
];
```

```
// — HELPERS —————
const peso = (n, currency = "₱") => `${currency}${Number(n || 0).toLocaleString("`
```

```
const num = (n) => Number(n || 0).toLocaleString("en-PH", { maximumFractionDigits
```

```
function getCostPerUnit(item) {
  const { purchasePrice, purchaseUnit, unit } = item;
  if (!purchasePrice || !purchaseUnit) return 0;
  const pu = (purchaseUnit || "").toLowerCase();
  if (unit === "g") {
    if (pu.includes("kg") || pu.includes("kilo")) return purchasePrice / 1000;
    if (pu.includes("500g")) return purchasePrice / 500;
    if (pu.includes("200g")) return purchasePrice / 200;
    const match = pu.match(/(\d+)g/); if (match) return purchasePrice / parseInt(
    return purchasePrice;
  }
  if (unit === "ml") {
    if (pu.includes("liter") || pu.includes("litre") || pu.includes("l")) return
    return purchasePrice;
  }
  if (unit === "pcs") {
    const match = pu.match(/(\d+)\s*pcs/);
    if (match) return purchasePrice / parseInt(match[1]);
    return purchasePrice;
  }
  return purchasePrice;
}
```

```
function computeRecord(record, items) {
  const item = items.find(i => i.id === record.itemId);
  if (!item) return { ...record, used: 0, usedCost: 0, costPerUnit: 0 };
  const used = Math.max(0, (record.beginning || 0) - (record.ending || 0));
  const costPerUnit = getCostPerUnit(item);
  return { ...record, used, usedCost: used * costPerUnit, costPerUnit };
}
```

```
function computePayrollRow(row) {
  return { ...row, totalCost: (row.dailyRate||0)+(row.allowance||0)+(row.sss||0)+
}
```

```
// — CONTEXT —————
```

```
const AppContext = createContext(null);
const useApp = () => useContext(AppContext);
```

```
// — MOCK AUTH —————
```

```
// In production, replace with Supabase auth:
// const { data: { user } } = await supabase.auth.getUser();
const MOCK_USERS = [
  { id: 1, email: "bryan@jscag.com", password: "admin123", name: "Bryan", role: "
  { id: 2, email: "maria@jscag.com", password: "maria123", name: "Maria Santos",
```

```

    { id: 3, email: "jose@jscag.com", password: "jose123", name: "Jose Cruz", role:
    { id: 4, email: "finance@jscag.com", password: "finance123", name: "Finance Tea
  ];

// — MAIN APP —————
export default function App() {
  const [user, setUser] = useState(null);
  const [page, setPage] = useState("dashboard");
  const [settings, setSettings] = useState(INITIAL_SETTINGS);
  const [items, setItems] = useState(SEED_ITEMS);
  const [categories, setCategories] = useState(SEED_CATEGORIES);
  const [products, setProducts] = useState(SEED_PRODUCTS);
  const [inventoryRecords, setInventoryRecords] = useState(SEED_INVENTORY);
  const [productionRecords, setProductionRecords] = useState(SEED_PRODUCTION);
  const [payrollRecords, setPayrollRecords] = useState(SEED_PAYROLL);
  const [overheadRecords, setOverheadRecords] = useState(SEED_OVERHEAD);
  const [toast, setToast] = useState(null);
  const [sidebarOpen, setSidebarOpen] = useState(true);

  // In production:
  // useEffect(() => { supabase.auth.getSession().then(({data:{session}}) => setU

function showToast(msg, type = "success") {
  setToast({ msg, type });
  setTimeout(() => setToast(null), 3500);
}

const canAccess = (requiredRole) => {
  if (!user) return false;
  const hierarchy = { admin: 4, finance: 3, auditor: 2, production: 1 };
  const roleAccess = { admin: ["admin"], finance: ["admin", "finance"], auditor
  return roleAccess[requiredRole]?.includes(user.role) ?? false;
};

const NAV = [
  { id: "dashboard", label: "Dashboard", icon: "■", roles: ["admin","auditor","
  { id: "inventory-list", label: "Item Master", icon: "📦", roles: ["admin"] },
  { id: "daily-inventory", label: "Daily Inventory", icon: "📅", roles: ["admin
  { id: "production", label: "Production Yield", icon: "🏭", roles: ["admin","p
  { id: "costing", label: "Product Costing", icon: "💰", roles: ["admin","finan
  { id: "payroll", label: "Payroll", icon: "👤", roles: ["admin","finance"] },
  { id: "overhead", label: "Overhead", icon: "🔦", roles: ["admin","finance"] }
  { id: "reports", label: "Reports", icon: "🇮🇹", roles: ["admin","finance","aud
  { id: "settings", label: "Settings", icon: "⚙️", roles: ["admin"] },
].filter(n => !user || n.roles.includes(user.role));

const ctx = { user, settings, setSettings, items, setItems, categories, setCate

```

```

if (!user) return <LoginPage onLogin={ (u) => setUser(u) } showToast={showToast}

return (
  <AppContext.Provider value={ctx}>
    <div style={{ display: "flex", minHeight: "100vh", fontFamily: "'DM Sans','
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;
      *{box-sizing:border-box;margin:0;padding:0}
      ::-webkit-scrollbar{width:5px;height:5px} ::-webkit-scrollbar-track{bac
      input,select,textarea{font-family:inherit}
      .nav-item{display:flex;align-items:center;gap:10px;padding:9px 14px;bor
      .nav-item:hover{background:#1e2235;color:#c8ccdc}
      .nav-item.active{background:#1a2a4a;color:#4d9fff}
      .card{background:#161924;border:1px solid #232840;border-radius:14px;pa
      .btn{padding:9px 18px;border-radius:8px;border:none;cursor:pointer;font
      .btn-primary{background:#2563eb;color:#fff} .btn-primary:hover{backgrou
      .btn-danger{background:#dc2626;color:#fff} .btn-danger:hover{background
      .btn-ghost{background:#1e2235;color:#8b92a8} .btn-ghost:hover{backgroun
      .btn-green{background:#065f46;color:#6ee7b7} .btn-green:hover{backgroun
      .btn-sm{padding:5px 12px;font-size:12px}
      .inp{background:#1e2235;border:1px solid #2a3050;color:#e8eaf0;border-r
      .inp:focus{border-color:#2563eb} .inp::placeholder{color:#4a5168}
      table{width:100%;border-collapse:collapse;font-size:13px}
      th{text-align:left;padding:10px 14px;background:#1a1d27;color:#6b7390;f
      td{padding:10px 14px;border-bottom:1px solid #1a1d27;color:#c4c9dc;vert
      tr:last-child td{border-bottom:none} tr:hover td{background:#0d1020}
      .badge{display:inline-block;padding:3px 9px;border-radius:20px;font-siz
      .badge-green{background:#0d2a1a;color:#34d399} .badge-red{background:#2
      .badge-blue{background:#0d1a3a;color:#60a5fa} .badge-yellow{background:
      label{font-size:11.5px;color:#6b7390;font-weight:600;text-transform:upp
      .stat-card{background:#161924;border:1px solid #232840;border-radius:14
      .stat-val{font-size:24px;font-weight:700;color:#e8eaf0;font-family:'DM
      .stat-label{font-size:11.5px;color:#6b7390;margin-top:4px;font-weight:5
      .section-title{font-size:20px;font-weight:700;color:#e8eaf0;margin-bott
      .section-sub{font-size:13px;color:#6b7390;margin-bottom:20px}
      .grid-2{display:grid;grid-template-columns:1fr 1fr;gap:16px}
      .grid-3{display:grid;grid-template-columns:1fr 1fr 1fr;gap:16px}
      .grid-4{display:grid;grid-template-columns:1fr 1fr 1fr 1fr;gap:16px}
      @media(max-width:900px){.grid-4{grid-template-columns:1fr 1fr}.grid-3{g
      @media(max-width:600px){.grid-2,.grid-3,.grid-4{grid-template-columns:1
      .modal-overlay{position:fixed;inset:0;background:rgba(0,0,0,0.75);displ
      .modal{background:#161924;border:1px solid #2a3050;border-radius:16px;p
      .cost-row{display:flex;justify-content:space-between;align-items:center
      .highlight-box{background:#0d1a3a;border:1px solid #1e3a6e;border-radiu
      .alert-warn{background:#2a1f0d;border:1px solid #78350f;color:#fbbf24;p
      .alert-info{background:#0d1a3a;border:1px solid #1e3a6e;color:#60a5fa;p

```

```

.tab-btn{padding:8px 18px;border-radius:8px;border:none;cursor:pointer;
.tab-btn.active{background:#1a2a4a;color:#4d9fff}
`</style>

{/* SIDEBAR */}
<div style={{ width: sidebarOpen ? 220 : 64, background: "#13161f", borde
  <div style={{ display: "flex", alignItems: "center", justifyContent: "s
    {sidebarOpen && <div>
      <div style={{ fontSize: 13, fontWeight: 800, color: "#2563eb", lett
        <div style={{ fontSize: 10, color: "#4a5168", marginTop: 1 }}>Commi
      </div>
      <button style={{ background: "none", border: "none", color: "#4a5168"
    </div>
  <nav style={{ marginTop: 12, flex: 1 }}>
    {NAV.map(n => (
      <button key={n.id} className={`nav-item ${page === n.id ? "active"
        <span style={{ fontSize: 15, flexShrink: 0 }}>{n.icon}</span>
        {sidebarOpen && <span>{n.label}</span>}
      </button>
    ))}
  </nav>
  <div style={{ borderTop: "1px solid #1e2235", paddingTop: 12 }}>
    {sidebarOpen && <div style={{ fontSize: 11, color: "#4a5168", marginB
      <div style={{ color: "#8b92a8", fontWeight: 600 }}>{user.name}</div>
      <div><span className="badge badge-blue" style={{ fontSize: 10, padd
    </div>
    <button className="nav-item" onClick={() => setUser(null)} title="Log
      <span style={{ fontSize: 15 }}><img alt="logout icon" data-bbox="555 565 565 575"/></span>
      {sidebarOpen && "Logout"}
    </button>
  </div>
</div>

{/* MAIN */}
<div style={{ flex: 1, padding: "28px 32px", overflowY: "auto", minWidth:
  {/* BREADCRUMB */}
  <div style={{ fontSize: 11, color: "#4a5168", marginBottom: 20, textTra
    {settings.companyName} / {NAV.find(n => n.id === page)?.label}
  </div>

  {page === "dashboard" && <Dashboard />}
  {page === "inventory-list" && <ItemMaster />}
  {page === "daily-inventory" && <DailyInventory />}
  {page === "production" && <ProductionYield />}
  {page === "costing" && <ProductCosting />}
  {page === "payroll" && <Payroll />}
  {page === "overhead" && <Overhead />}

```

```

        {page === "reports" && <Reports />}
        {page === "settings" && <Settings />}
    </div>

    {/* TOAST */}
    {toast && <div style={{ position: "fixed", bottom: 24, right: 24, backgro
        {toast.msg}
    </div>}
    </div>
    </AppContext.Provider>
    );
}

// — LOGIN PAGE —————
function LoginPage({ onLogin, showToast, toast, settings }) {
    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");
    const [loading, setLoading] = useState(false);

    function handleLogin() {
        setLoading(true);
        // In production: const { data, error } = await supabase.auth.signInWithPassw
        setTimeout(() => {
            const found = MOCK_USERS.find(u => u.email === email && u.password === pass
            if (found) { onLogin(found); }
            else { showToast("Mali ang email o password.", "error"); }
            setLoading(false);
        }, 600);
    }

    return (
        <div style={{ minHeight: "100vh", background: "#0f1117", display: "flex", ali
        <style>{@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght
        <div style={{ width: "100%", maxWidth: 400 }}>
            <div style={{ textAlign: "center", marginBottom: 40 }}>
                <div style={{ fontSize: 32, fontWeight: 800, color: "#2563eb", letterSp
                <div style={{ fontSize: 13, color: "#6b7390", marginTop: 4 }}>Commissar
            </div>
            <div style={{ background: "#161924", border: "1px solid #232840", borderR
                <div style={{ fontWeight: 700, fontSize: 18, marginBottom: 24, color: "
                <div style={{ marginBottom: 16 }}>
                    <label>Email</label>
                    <input className="inp" type="email" placeholder="bryan@jscag.com" val
                </div>
                <div style={{ marginBottom: 24 }}>
                    <label>Password</label>
                    <input className="inp" type="password" placeholder="....." value={

```

```

    </div>
    <button onClick={handleLogin} disabled={loading} style={{ width: "100%"
      {loading ? "Loading..." : "Pumasok"}}
    </button>
    <div style={{ marginTop: 20, background: "#1a1d27", borderRadius: 8, pa
      <div style={{ fontSize: 11, color: "#4a5168", marginBottom: 6, fontWe
        {[["bryan@jscag.com", "admin123", "Admin"], ["maria@jscag.com", "maria123
          <div key={e} style={{ fontSize: 11, color: "#6b7390", cursor: "poin
            <span style={{ color: "#4d9fff" }}>{r}</span>: {e} / {p}
          </div>
        )}}
      </div>
    </div>
    {toast && <div style={{ marginTop: 16, background: "#7f1d1d", border: "1p
    </div>
  </div>
);
}

```

```

// — DASHBOARD —————
function Dashboard() {
  const { inventoryRecords, productionRecords, payrollRecords, overheadRecords, i
  const inv = inventoryRecords.filter(r => r.date === TODAY).map(r => computeReco
  const prod = productionRecords.filter(r => r.date === TODAY);
  const pay = payrollRecords.filter(r => r.date === TODAY && r.present).map(compu
  const oh = overheadRecords.filter(r => r.date === TODAY);

  const totalIng = inv.reduce((s, r) => s + r.usedCost, 0);
  const totalPay = pay.reduce((s, r) => s + r.totalCost, 0);
  const totalOH = oh.reduce((s, r) => s + r.amount, 0);
  const totalProd = totalIng + totalPay + totalOH;
  const goodPacks = prod.reduce((s, r) => s + r.goodPacks, 0);
  const costPerPack = goodPacks > 0 ? totalProd / goodPacks : 0;

  const lowStock = items.filter(item => {
    const rec = [...inventoryRecords].filter(r => r.itemId === item.id).sort((a,b
    return rec && rec.ending <= item.reorderLevel;
  });

  const topIng = [...inv].sort((a,b) => b.usedCost - a.usedCost).slice(0, 6);
  const weekRejects = productionRecords.reduce((s,r) => s + r.rejects, 0);
  const weekGood = productionRecords.reduce((s,r) => s + r.goodPacks, 0);

  return (
    <div>
      <div className="section-title">Dashboard</div>
      <div className="section-sub">{new Date(TODAY+"T00:00:00").toLocaleDateStrin

```

```

<div className="grid-4" style={{ marginBottom: 18 }}>
  [[
    { label: "Ingredient Cost", value: peso(totalIng), color: "#e74c3c", ic
    { label: "Payroll Cost", value: peso(totalPay), color: "#3498db", icon:
    { label: "Overhead Cost", value: peso(totalOH), color: "#9b59b6", icon:
    { label: "Total Prod. Cost", value: peso(totalProd), color: "#f39c12",
  ].map(s => (
    <div key={s.label} className="stat-card">
      <div style={{ fontSize: 20, marginBottom: 8 }}>{s.icon}</div>
      <div className="stat-val" style={{ color: s.color, fontSize: 20 }}>{s
      <div className="stat-label">{s.label}</div>
    </div>
  )))
</div>

```

```

<div className="grid-3" style={{ marginBottom: 18 }}>
  <div className="highlight-box">
    <div style={{ fontSize: 12, color: "#6b7390", marginBottom: 6 }}>Good P
    <div style={{ fontSize: 30, fontWeight: 700, color: "#34d399", fontFami
    <div style={{ fontSize: 11, color: "#4a5168", marginTop: 4 }}>packs pro
  </div>
  <div className="highlight-box">
    <div style={{ fontSize: 12, color: "#6b7390", marginBottom: 6 }}>Cost P
    <div style={{ fontSize: 30, fontWeight: 700, color: "#60a5ff", fontFami
    <div style={{ fontSize: 11, color: "#4a5168", marginTop: 4 }}>total ÷ g
  </div>
  <div className="highlight-box">
    <div style={{ fontSize: 12, color: "#6b7390", marginBottom: 6 }}>Reject
    <div style={{ fontSize: 30, fontWeight: 700, color: weekGood+weekReject
      {weekGood+weekRejects > 0 ? ((weekRejects/(weekGood+weekRejects))*100
    </div>
    <div style={{ fontSize: 11, color: "#4a5168", marginTop: 4 }}>threshold
  </div>
</div>

```

```

<div className="grid-2">
  <div className="card">
    <div style={{ fontWeight: 700, marginBottom: 14, color: lowStock.length
      {lowStock.length > 0 ? `⚠️ Low Stock (${lowStock.length})` : "✅ Stoc
    </div>
    {lowStock.length === 0
      ? <div style={{ color: "#4a5168", fontSize: 13 }}>Lahat ng items ay n
      : lowStock.map(item => {
        const rec = [...inventoryRecords].filter(r => r.itemId === item.id)
        return <div key={item.id} className="alert-warn" style={{ marginBot
          <strong>{item.name}</strong> - Stock: {rec?.ending} {item.unit} |

```

```

        </div>;
    })
  }
</div>
<div className="card">
  <div style={{ fontWeight: 700, marginBottom: 14, fontSize: 14 }}>🔥 Top
  {topIng.length === 0
    ? <div style={{ color: "#4a5168", fontSize: 13 }}>Walang inventory re
    : topIng.map(r => {
      const item = items.find(i => i.id === r.itemId);
      const pct = totalIng > 0 ? (r.usedCost / totalIng * 100).toFixed(0)
      return <div key={r.id}>
        <div style={{ display: "flex", justifyContent: "space-between", f
          <span style={{ color: "#c4c9dc" }}>{item?.name}</span>
          <span style={{ fontFamily: "'DM Mono',monospace", color: "#60a5
        </div>
        <div style={{ height: 4, background: "#1a1d27", borderRadius: 2,
          <div style={{ height: "100%", width: pct + "%", background: "#2
        </div>
      </div>;
    })
  }
</div>
</div>
</div>
);
}

```

```

// — ITEM MASTER —————
function ItemMaster() {
  const { items, setItems, categories, peso, showToast, getCostPerUnit, exportToE
  const [showForm, setShowForm] = useState(false);
  const [editing, setEditing] = useState(null);
  const [filter, setFilter] = useState("");
  const [catFilter, setCatFilter] = useState("");
  const blank = { name: "", categoryId: 1, unit: "g", purchaseUnit: "1 kg", purch
  const [form, setForm] = useState(blank);

  // Supabase: const {data} = await supabase.from('inventory_items').select('*');
  const filtered = items.filter(i =>
    i.name.toLowerCase().includes(filter.toLowerCase()) &&
    (catFilter === "" || i.categoryId === Number(catFilter))
  );

  function save() {
    if (!form.name.trim()) return;
    if (editing) {

```

```

    // supabase.from('inventory_items').update(form).eq('id', editing)
    setItems(items.map(i => i.id === editing ? { ...form, id: editing } : i));
    showToast("Item na-update.");
  } else {
    // supabase.from('inventory_items').insert(form)
    setItems([...items, { ...form, id: Date.now() }]);
    showToast("Item na-add.");
  }
  setShowForm(false);
}

function doExport() {
  exportToExcel(filtered.map(i => ({
    Name: i.name, Category: categories.find(c => c.id === i.categoryId)?.name,
    Unit: i.unit, "Purchase Unit": i.purchaseUnit, "Purchase Price": i.purchase
    "Cost/Unit": getCostPerUnit(i).toFixed(6), Supplier: i.supplier,
    "Reorder Level": i.reorderLevel, Status: i.active ? "Active" : "Inactive"
  })), "Item Master", "JSCAG-Item-Master");
}

return (
  <div>
    <div style={{ display: "flex", justifyContent: "space-between", alignItems:
      <div><div className="section-title">Item Master List</div><div className=
        <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
          <button className="btn btn-green btn-sm" onClick={doExport}><img alt="Download icon" data-bbox="785 525 805 545"/> Export</
          <button className="btn btn-primary" onClick={() => { setForm(blank); se
        </div>
      </div>

    <div style={{ display: "flex", gap: 10, marginBottom: 14, flexWrap: "wrap"
      <input className="inp" style={{ maxWidth: 240 }} placeholder="Hanapin ang
      <select className="inp" style={{ width: "auto" }} value={catFilter} onCha
        <option value="">Lahat ng Categories</option>
        {categories.map(c => <option key={c.id} value={c.id}>{c.name}</option>
      </select>
    </div>

    <div className="card" style={{ padding: 0, overflow: "hidden" }}>
      <table>
        <thead><tr><th>Item</th><th>Category</th><th>Unit</th><th>Purchase Unit
        <tbody>
          {filtered.map(item => {
            const cat = categories.find(c => c.id === item.categoryId);
            return <tr key={item.id}>
              <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{item.name}</td>
              <td><span className="badge" style={{ background: cat?.color + "22

```

```

        <td style={{ fontFamily: "'DM Mono',monospace", color: "#6b7390"
        <td style={{ color: "#6b7390" }}>{item.purchaseUnit}</td>
        <td style={{ fontFamily: "'DM Mono',monospace" }}>{peso(item.purc
        <td style={{ fontFamily: "'DM Mono',monospace", color: "#60a5ff",
        <td style={{ fontFamily: "'DM Mono',monospace" }}>{item.reorderLe
        <td style={{ color: "#6b7390", fontSize: 12 }}>{item.supplier}</t
        <td><span className={`badge ${item.active ? "badge-green" : "badg
        <td><button className="btn btn-ghost btn-sm" onClick={() => { set
        </tr>;
    }}}
    {filtered.length === 0 && <tr><td colspan={10} style={{ textAlign: "c
    </tbody>
  </table>
</div>

{showForm && <FormModal title={editing ? "I-Edit ang Item" : "Mag-add ng It
  <div className="grid-2" style={{ gap: 12 }}>
    <div style={{ gridColumn: "span 2" }}><label>Item Name *</label><input
    <div><label>Category</label>
      <select className="inp" value={form.categoryId} onChange={e => setFor
        {categories.map(c => <option key={c.id} value={c.id}>{c.name}</opti
      </select>
    </div>
    <div><label>Unit</label>
      <select className="inp" value={form.unit} onChange={e => setForm({...
        {["g","kg","ml","l","pcs","pack","box"].map(u => <option key={u}>{u
      </select>
    </div>
    <div><label>Purchase Unit (paano binibili)</label><input className="inp
    <div><label>Purchase Price (₱)</label><input className="inp" type="numb
    <div><label>Reorder Level</label><input className="inp" type="number" v
    <div><label>Supplier</label><input className="inp" value={form.supplier
    <div><label>Status</label>
      <select className="inp" value={form.active?"1":"0"} onChange={e => se
        <option value="1">Active</option><option value="0">Inactive</option
      </select>
    </div>
    <div style={{ gridColumn: "span 2" }}><label>Notes</label><input classN
  </div>
  {form.purchasePrice > 0 && <div className="alert-info" style={{ marginTop
    Auto-compute: {peso(getCostPerUnit(form))} per {form.unit}
  </div>}
</FormModal>
</div>
);
}

```

```
// — DAILY INVENTORY —————
function DailyInventory() {
  const { items, categories, inventoryRecords, setInventoryRecords, showToast, co
  const [date, setDate] = useState(TODAY);
  const [showForm, setShowForm] = useState(false);
  const [form, setForm] = useState({ itemId: items[0]?.id, beginning: 0, ending:

  const records = inventoryRecords.filter(r => r.date === date).map(r => computeR
  const totalUsedCost = records.reduce((s,r) => s + r.usedCost, 0);

  function autoCarryOver() {
    const prev = new Date(date); prev.setDate(prev.getDate()-1);
    const yStr = prev.toISOString().split("T")[0];
    const prevRecs = inventoryRecords.filter(r => r.date === yStr);
    let added = 0;
    const newRecs = [...inventoryRecords];
    prevRecs.forEach(pr => {
      if (!inventoryRecords.find(r => r.date === date && r.itemId === pr.itemId))
        newRecs.push({...pr, id: Date.now()+Math.random(), date, beginning: pr.en
        added++;
    });
    setInventoryRecords(newRecs);
    showToast(`${added} item(s) na-carry over mula ${yStr}.`);
  }

  function addRecord() {
    const item = items.find(i => i.id === Number(form.itemId));
    if (!item) return;
    if (inventoryRecords.find(r => r.date === date && r.itemId === Number(form.it
      showToast("Naka-record na ang item na ito para sa date na ito.", "error");
    }
    const cpu = getCostPerUnit(item);
    setInventoryRecords([...inventoryRecords, { ...form, id: Date.now(), date, it
    showToast("Record na-add.");
    setShowForm(false);
  }

  function doExport() {
    exportToExcel(records.map(r => {
      const item = items.find(i => i.id === r.itemId);
      return { Date: date, Item: item?.name, Beginning: r.beginning, Ending: r.en
    })), "Inventory", `JSCAG-Inventory-${date}`);
  }

  return (
    <div>
```

```

<div style={{ display: "flex", justifyContent: "space-between", alignItems:
  <div><div className="section-title">Daily Inventory</div><div className="
  <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
    <input type="date" className="inp" style={{ width: "auto" }} value={dat
    <button className="btn btn-ghost btn-sm" onClick={autoCarryOver}><img alt="
    <button className="btn btn-green btn-sm" onClick={doExport}><img alt="
    <button className="btn btn-primary" onClick={() => setShowForm(true)}>+
  </div>
</div>

<div className="grid-3" style={{ marginBottom: 16 }}>
  <div className="stat-card"><div className="stat-val">{records.length}</di
  <div className="stat-card"><div className="stat-val" style={{ color: "#f8
  <div className="stat-card"><div className="stat-val" style={{ color: "#60
</div>

<div className="card" style={{ padding: 0, overflow: "hidden" }}>
  <table>
    <thead><tr><th>Item</th><th>Category</th><th>Beginning</th><th>Ending <img alt="
    <tbody>
      {records.length === 0 && <tr><td colspan={8} style={{ textAlign: "cen
      {records.map(r => {
        const item = items.find(i => i.id === r.itemId);
        const cat = categories.find(c => c.id === r.categoryId);
        return <tr key={r.id}>
          <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{item?.name}</t
          <td><span className="badge" style={{ background: cat?.color+"22",
          <td style={{ fontFamily: "'DM Mono',monospace" }}>{num(r.beginnin
          <td>
            <div style={{ display: "flex", alignItems: "center", gap: 6 }}>
              <input type="number" className="inp" style={{ width: 90, padd
              onChange={e => setInventoryRecords(inventoryRecords.map(ir
              <span style={{ color: "#6b7390", fontSize: 11 }}>{r.unit}</sp
            </div>
          </td>
          <td style={{ fontFamily: "'DM Mono',monospace", color: r.used > 0
          <td style={{ fontFamily: "'DM Mono',monospace", color: "#6b7390"
          <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700,
          <td style={{ color: "#6b7390", fontSize: 12 }}>{r.encoder}</td>
        </tr>;
      })}
      {records.length > 0 && <tr style={{ background: "#1a1d27" }}>
        <td colspan={6} style={{ textAlign: "right", fontWeight: 700, color
        <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700, co
        <td></td>
      </tr>}
    </tbody>

```

```

        </table>
      </div>

      {showForm && <FormModal title="Mag-add ng Inventory Record" onClose={() =>
        <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
          <div><label>Item</label>
            <select className="inp" value={form.itemId} onChange={e => setForm({.
              {items.filter(i=>i.active).map(i => <option key={i.id} value={i.id}
            </select>
          </div>
          <div className="grid-2" style={{ gap: 12 }}>
            <div><label>Beginning</label><input className="inp" type="number" val
              <div><label>Ending</label><input className="inp" type="number" value=
            </div>
            <div><label>Encoder (Pangalan mo)</label><input className="inp" value={
              <div><label>Notes</label><input className="inp" value={form.notes} onCh
            </div>
          </FormModal>}
        </div>
      );
    }
  }

```

```

// — PRODUCTION YIELD —————
function ProductionYield() {
  const { products, productionRecords, setProductionRecords, showToast, num, expo
  const [date, setDate] = useState(TODAY);
  const [showForm, setShowForm] = useState(false);
  const [form, setForm] = useState({ productId: products[0]?.id, batchNo: "", goo

  const records = productionRecords.filter(r => r.date === date);
  const totalGood = records.reduce((s,r) => s+r.goodPacks, 0);
  const totalRejects = records.reduce((s,r) => s+r.rejects, 0);
  const rejectRate = totalGood+totalRejects > 0 ? (totalRejects/(totalGood+totalR

  function doExport() {
    exportToExcel(records.map(r => {
      const prod = products.find(p => p.id === r.productId);
      return { Date: date, Product: prod?.name, "Batch No": r.batchNo, "Good Pack
    })), "Production", `JSCAG-Production-${date}`);
  }

  return (
    <div>
      <div style={{ display: "flex", justifyContent: "space-between", alignItems:
        <div><div className="section-title">Production Yield</div><div className=
        <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
          <input type="date" className="inp" style={{ width: "auto" }} value={dat

```

```

        <button className="btn btn-green btn-sm" onClick={doExport}><img alt="Download icon" data-bbox="785 48 805 65"/> Export</button>
        <button className="btn btn-primary" onClick={() => setShowForm(true)}>+
    </div>
</div>

<div className="grid-3" style={{ marginBottom: 16 }}>
    <div className="stat-card"><div className="stat-val" style={{ color: "#34
    <div className="stat-card"><div className="stat-val" style={{ color: "#f8
    <div className="stat-card">
        <div className="stat-val" style={{ color: rejectRate > settings.rejectR
        <div className="stat-label">Reject Rate {rejectRate > settings.rejectRa
    </div>
</div>

<div className="card" style={{ padding: 0, overflow: "hidden" }}>
    <table>
        <thead><tr><th>Product</th><th>Batch</th><th>Good Packs</th><th>Rejects
    <tbody>
        {records.length === 0 && <tr><td colspan={7} style={{ textAlign: "cen
        {records.map(r => {
            const prod = products.find(p => p.id === r.productId);
            const total = r.goodPacks + r.rejects;
            const rr = total > 0 ? (r.rejects/total*100).toFixed(1) : 0;
            return <tr key={r.id}>
                <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{prod?.name}</td>
                <td style={{ fontFamily: "'DM Mono',monospace", color: "#6b7390"
                <td style={{ fontFamily: "'DM Mono',monospace", color: "#34d399",
                <td style={{ fontFamily: "'DM Mono',monospace", color: "#f87171"
                <td style={{ fontFamily: "'DM Mono',monospace" }}>{num(total)}</td>
                <td><span className={`badge ${rr > settings.rejectRateThreshold ?
                <td style={{ color: "#6b7390", fontSize: 12 }}>{r.encoder}</td>
            </tr>;
        })}
    </tbody>
</table>
</div>

{showForm && <FormModal title="Mag-add ng Production Yield" onClose={() =>
    setProductionRecords([...productionRecords, {...form, id: Date.now(), dat
    showToast("Production record na-add."); setShowForm(false);
}}>
    <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
        <div><label>Product</label>
        <select className="inp" value={form.productId} onChange={e => setForm
            {products.map(p => <option key={p.id} value={p.id}>{p.name}</option>
        </select>
    </div>

```

```

    <div className="grid-2" style={{ gap: 12 }}>
      <div><label>Batch No.</label><input className="inp" value={form.batch}
      <div><label>Encoder</label><input className="inp" value={form.encoder}
      <div><label>Good Packs</label><input className="inp" type="number" va
      <div><label>Rejects</label><input className="inp" type="number" value
    </div>
    <div><label>Notes</label><input className="inp" value={form.notes} onCh
  </div>
  </FormModal>
</div>
);
}

```

// — PRODUCT COSTING —————

```

function ProductCosting() {
  const { inventoryRecords, productionRecords, payrollRecords, overheadRecords, i
  const [date, setDate] = useState(TODAY);

  const invRecs = inventoryRecords.filter(r => r.date === date).map(r => computeR
  const prodRecs = productionRecords.filter(r => r.date === date);
  const payRecs = payrollRecords.filter(r => r.date === date && r.present).map(co
  const ohRecs = overheadRecords.filter(r => r.date === date);

  const getCat = (r) => { const item = items.find(i => i.id === r.itemId); return
  const totalKitchen = invRecs.filter(r => getCat(r) === "Kitchen").reduce((s,r)
  const totalPkg = invRecs.filter(r => getCat(r) === "Packaging").reduce((s,r) =>
  const totalClean = invRecs.filter(r => getCat(r) === "Cleaning / Sanitation").r
  const totalOtherInv = invRecs.filter(r => ![ "Kitchen","Packaging","Cleaning / S
  const totalPay = payRecs.reduce((s,r) => s+r.totalCost, 0);
  const totalOH = ohRecs.reduce((s,r) => s+r.amount, 0);
  const grandTotal = totalKitchen + totalPkg + totalClean + totalOtherInv + total
  const goodPacks = prodRecs.reduce((s,r) => s+r.goodPacks, 0);
  const costPerPack = goodPacks > 0 ? grandTotal / goodPacks : 0;

  const firstProd = products.find(p => prodRecs.find(r => r.productId === p.id));
  const margin = firstProd ? firstProd.sellingPrice - costPerPack : 0;
  const marginPct = firstProd?.sellingPrice > 0 ? (margin/firstProd.sellingPrice*

  function doExport() {
    exportToExcel([
      { Category: "Kitchen / Ingredients", Amount: totalKitchen },
      { Category: "Packaging", Amount: totalPkg },
      { Category: "Cleaning / Sanitation", Amount: totalClean },
      { Category: "Other Materials", Amount: totalOtherInv },
      { Category: "Payroll", Amount: totalPay },
      { Category: "Overhead", Amount: totalOH },
      { Category: "TOTAL", Amount: grandTotal },
    ]
  )
}

```

```

    { Category: "Good Packs", Amount: goodPacks },
    { Category: "Cost Per Pack", Amount: costPerPack },
  ], "Costing", `JSCAG-Costing-${date}`);
}

return (
  <div>
    <div style={{ display: "flex", justifyContent: "space-between", alignItems:
      <div><div className="section-title">Product Costing</div><div className="
      <div style={{ display: "flex", gap: 8 }}>
        <input type="date" className="inp" style={{ width: "auto" }} value={dat
        <button className="btn btn-green btn-sm" onClick={doExport}><img alt="Download icon" data-bbox="788 258 805 275"/> Export</
      </div>
    </div>

    <div className="grid-2">
      <div className="card">
        <div style={{ fontWeight: 700, fontSize: 14, marginBottom: 14 }}><img alt="Shopping cart icon" data-bbox="835 375 855 390"/> Inv
        {invRecs.length === 0 ? <div style={{ color: "#4a5168", fontSize: 13 }}>
          : invRecs.map(r => {
            const item = items.find(i => i.id === r.itemId);
            const cat = categories.find(c => c.id === item?.categoryId);
            return <div key={r.id} className="cost-row">
              <span>
                <span style={{ color: "#e8eaf0" }}>{item?.name}</span>
                <span style={{ display: "block", fontSize: 10.5, color: "#4a5168" }}>
              </span>
                <span style={{ fontFamily: "'DM Mono', monospace", color: "#c4c9dc" }}>
              </div>;
            })
          }
        </div>

        <div className="card">
          <div style={{ fontWeight: 700, fontSize: 14, marginBottom: 14 }}><img alt="Money bag icon" data-bbox="835 695 855 715"/> Cos
          {[
            { label: "Kitchen / Ingredients", value: totalKitchen, color: "#e74c3c" },
            { label: "Packaging", value: totalPkg, color: "#3498db" },
            { label: "Cleaning / Sanitation", value: totalClean, color: "#2ecc71" },
            { label: "Other Materials", value: totalOtherInv, color: "#95a5a6" },
            { label: "Payroll", value: totalPay, color: "#9b59b6" },
            { label: "Overhead", value: totalOH, color: "#f39c12" },
          ]}.map(row => <div key={row.label} className="cost-row">
            <span style={{ display: "flex", alignItems: "center", gap: 8 }}>
              <span style={{ width: 8, height: 8, borderRadius: "50%", background
              <span style={{ color: "#c4c9dc" }}>{row.label}</span>
            </span>

```

```

        <span style={{ fontFamily: "'DM Mono',monospace", color: "#e8eaf0", f
    </div>}}

    <div style={{ display: "flex", justifyContent: "space-between", padding
        <span>Total Daily Cost</span>
        <span style={{ fontFamily: "'DM Mono',monospace", color: "#fbbf24" }}
    </div>

    <div style={{ marginTop: 16, borderTop: "1px solid #1e2235", paddingTop
        <div style={{ fontSize: 12, color: "#6b7390", marginBottom: 6 }}>Good
        <div style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700, fon
    </div>

    <div className="highlight-box" style={{ marginTop: 16 }}>
        <div style={{ fontSize: 12, color: "#6b7390", marginBottom: 6 }}>COST
        <div style={{ fontSize: 36, fontWeight: 800, color: "#60a5ff", fontFa
        <div style={{ fontSize: 11, color: "#4a5168", marginTop: 4 }}>{peso(g
        {firstProd && goodPacks > 0 && <div style={{ marginTop: 10, fontSize:
            Selling: {peso(firstProd.sellingPrice)} | Gross Margin: {peso(margi
        </div>}
        </div>
    </div>
</div>
</div>
    );
}

// — PAYROLL —————
function Payroll() {
    const { payrollRecords, setPayrollRecords, showToast, peso, computePayrollRow,
    const [date, setDate] = useState(TODAY);
    const [showForm, setShowForm] = useState(false);
    const [form, setForm] = useState({ employee: "", role: "Production Staff", dail

    const records = payrollRecords.filter(r => r.date === date).map(computePayrollR
    const totalCost = records.filter(r => r.present).reduce((s,r) => s+r.totalCost,

    function doExport() {
        exportToExcel(records.map(r => ({
            Date: date, Employee: r.employee, Role: r.role, "Daily Rate": r.dailyRate,
            Allowance: r.allowance, SSS: r.sss, PhilHealth: r.philhealth, "Pag-IBIG": r
            "Other Benefits": r.otherBenefits, "Total Cost": r.totalCost, Present: r.pr
        })), "Payroll", `JSCAG-Payroll-${date}`);
    }

    return (
        <div>

```

```

<div style={{ display: "flex", justifyContent: "space-between", alignItems:
  <div><div className="section-title">Payroll</div><div className="section-
  <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
    <input type="date" className="inp" style={{ width: "auto" }} value={dat
    <button className="btn btn-green btn-sm" onClick={doExport}><img alt="Download icon" data-bbox="788 125 805 142"/> Export</
    <button className="btn btn-primary" onClick={() => setShowForm(true)}>+
  </div>
</div>

<div className="grid-3" style={{ marginBottom: 16 }}>
  <div className="stat-card"><div className="stat-val">{records.filter(r=>r
  <div className="stat-card"><div className="stat-val" style={{ color: "#f8
  <div className="stat-card"><div className="stat-val" style={{ color: "#9b
</div>

<div className="card" style={{ padding: 0, overflow: "hidden" }}>
  <table>
    <thead><tr><th>Employee</th><th>Role</th><th>Daily Rate</th><th>Allowan
    <tbody>
      {records.length === 0 && <tr><td colspan={7} style={{ textAlign: "cen
      {records.map(r => <tr key={r.id} style={{ opacity: r.present ? 1 : 0.
        <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{r.employee}</td>
        <td style={{ color: "#6b7390" }}>{r.role}</td>
        <td style={{ fontFamily: "'DM Mono',monospace" }}>{peso(r.dailyRate
        <td style={{ fontFamily: "'DM Mono',monospace" }}>{peso(r.allowance
        <td style={{ fontFamily: "'DM Mono',monospace", color: "#6b7390" }}
        <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700, co
        <td>
          <button className={`badge ${r.present ? "badge-green" : "badge-re
            onClick={() => setPayrollRecords(payrollRecords.map(pr => pr.id
              {r.present ? "Present" : "Absent"}
          </button>
        </td>
      </tr>)}
    </tbody>
  </table>
</div>

{showForm && <FormModal title="Mag-add ng Employee" onClose={() => setShowF
  if (!form.employee) return;
  setPayrollRecords([...payrollRecords, {...form, id: Date.now(), date, dai
  showToast("Payroll record na-add."); setShowForm(false);
}}>
  <div className="grid-2" style={{ gap: 12 }}>
    <div style={{ gridColumn: "span 2" }}><label>Pangalan ng Employee *</la
    <div><label>Role</label>
    <select className="inp" value={form.role} onChange={e => setForm({...

```

```

        {"Production Staff","Packing Staff","Inventory Auditor","Finance",
    </select>
</div>
<div><label>Daily Rate (₱)</label><input className="inp" type="number"
<div><label>Allowance (₱)</label><input className="inp" type="number" v
<div><label>SSS (₱)</label><input className="inp" type="number" value={
<div><label>PhilHealth (₱)</label><input className="inp" type="number"
<div><label>Pag-IBIG (₱)</label><input className="inp" type="number" va
<div><label>Other Benefits (₱)</label><input className="inp" type="numb
{form.dailyRate > 0 && <div style={{ gridColumn: "span 2" }} className=
    Total: {peso((parseFloat(form.dailyRate)||0)+(parseFloat(form.allowan
</div>}
</div>
</FormModal>}
</div>
);
}

```

```

// — OVERHEAD —————
function Overhead() {
    const { overheadRecords, setOverheadRecords, showToast, peso, settings, exportT
    const [date, setDate] = useState(TODAY);
    const [showForm, setShowForm] = useState(false);
    const [monthly, setMonthly] = useState(30000);
    const [form, setForm] = useState({ costType: "Electricity", amount: 0, allocati
    const COST_TYPES = ["Electricity","Water","Gas / LPG","Rent","Maintenance","Tra

    const records = overheadRecords.filter(r => r.date === date);
    const total = records.reduce((s,r) => s+r.amount, 0);

    function addRecord() {
        setOverheadRecords([...overheadRecords, {...form, id: Date.now(), date, amoun
        showToast("Overhead record na-add."); setShowForm(false);
    }

    return (
        <div>
            <div style={{ display: "flex", justifyContent: "space-between", alignItems:
                <div><div className="section-title">Overhead Costs</div><div className="s
                <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
                    <input type="date" className="inp" style={{ width: "auto" }} value={dat
                    <button className="btn btn-green btn-sm" onClick={() => exportToExcel(r
                    <button className="btn btn-primary" onClick={() => setShowForm(true)}>+
                </div>
            </div>

            <div className="grid-2" style={{ marginBottom: 16 }}>

```

```

    <div className="stat-card"><div className="stat-val">{records.length}</div>
    <div className="stat-card"><div className="stat-val" style={{ color: "#f3
</div>

<div className="card" style={{ padding: 0, overflow: "hidden", marginBottom
  <table>
    <thead><tr><th>Cost Type</th><th>Amount</th><th>Allocation</th><th>Note
    <tbody>
      {records.length === 0 && <tr><td colSpan={4} style={{ textAlign: "cen
      {records.map(r => <tr key={r.id}>
        <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{r.costType}</td>
        <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700, co
        <td><span className="badge badge-yellow">{r.allocationType}</span><
        <td style={{ color: "#6b7390" }}>{r.notes || "-"}</td>
      </tr>)}
      {records.length > 0 && <tr style={{ background: "#1a1d27" }}>
        <td style={{ fontWeight: 700, color: "#e8eaf0" }}>Total</td>
        <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700, co
        <td colSpan={2}></td>
      </tr>
    </tbody>
  </table>
</div>

<div className="card">
  <div style={{ fontWeight: 700, fontSize: 14, marginBottom: 14 }}>💡 Month
  <div style={{ display: "flex", alignItems: "center", gap: 16, flexWrap: "
    <div style={{ flex: 1, minWidth: 180 }}>
      <label>Monthly Amount (₹)</label>
      <input className="inp" type="number" value={monthly} onChange={e => s
    </div>
    <div style={{ fontSize: 20, color: "#4a5168", paddingTop: 16 }}>→</div>
    <div style={{ flex: 1 }}>
      <div style={{ fontSize: 12, color: "#6b7390", marginBottom: 4 }}>DAIL
      <div style={{ fontFamily: "'DM Mono',monospace", fontSize: 24, fontWe
      <div style={{ fontSize: 11, color: "#4a5168" }}>÷ {settings.workingDa
    </div>
  </div>
</div>

{showForm && <FormModal title="Mag-add ng Overhead Cost" onClose={() => set
  <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
    <div><label>Cost Type</label>
    <select className="inp" value={form.costType} onChange={e => setForm(
      {COST_TYPES.map(t => <option key={t}>{t}</option>)}
    </select>
  </div>

```

```

    <div><label>Allocation Type</label>
      <select className="inp" value={form.allocationType} onChange={e => se
        <option value="daily">Daily</option>
        <option value="weekly">Weekly</option>
        <option value="monthly">Monthly</option>
      </select>
    </div>
    <div>
      <label>Amount (₹) {form.allocationType !== "daily" && `– Daily equiv:
      <input className="inp" type="number" value={form.amount} onChange={e
    </div>
    <div><label>Notes</label><input className="inp" value={form.notes} onCh
    </div>
  </FormModal>
</div>
);
}

```

// — REPORTS —————

```

function Reports() {
  const { inventoryRecords, productionRecords, payrollRecords, overheadRecords, i
  const [tab, setTab] = useState("costing");
  const [dateFrom, setDateFrom] = useState(YESTERDAY);
  const [dateTo, setDateTo] = useState(TODAY);

  const dates = [];
  let cur = new Date(dateFrom+"T00:00:00"), endD = new Date(dateTo+"T00:00:00");
  while (cur <= endD) { dates.push(cur.toISOString().split("T")[0]); cur.setDate(

  const costingRows = dates.map(date => {
    const inv = inventoryRecords.filter(r => r.date === date).map(r => computeRec
    const prod = productionRecords.filter(r => r.date === date);
    const pay = payrollRecords.filter(r => r.date === date && r.present).map(comp
    const oh = overheadRecords.filter(r => r.date === date);
    const totalI = inv.reduce((s,r) => s+r.usedCost, 0);
    const totalP = pay.reduce((s,r) => s+r.totalCost, 0);
    const totalO = oh.reduce((s,r) => s+r.amount, 0);
    const total = totalI+totalP+totalO;
    const packs = prod.reduce((s,r) => s+r.goodPacks, 0);
    const rejects = prod.reduce((s,r) => s+r.rejects, 0);
    return { date, "Ingredient Cost": totalI, "Payroll": totalP, "Overhead": tota
  });

  const TABS = [
    { id: "costing", label: "Cost Per Pack History" },
    { id: "inventory", label: "Inventory Usage" },
    { id: "payroll", label: "Payroll Report" },
  ]
}

```

```

    { id: "overhead", label: "Overhead Report" },
  ];

return (
  <div>
    <div className="section-title">Reports</div>
    <div className="section-sub">I-filter at i-export ang historical data.</div>

    <div style={{ display: "flex", gap: 8, marginBottom: 18, flexWrap: "wrap" }
      {TABS.map(t => <button key={t.id} className={`tab-btn ${tab === t.id ? "a
    </div>

    <div style={{ display: "flex", gap: 12, marginBottom: 18, flexWrap: "wrap",
      <div><label>Mula sa</label><input type="date" className="inp" style={{ wi
    <div><label>Hanggang</label><input type="date" className="inp" style={{ w
    <button className="btn btn-green" onClick={() => exportToExcel(
      tab === "costing" ? costingRows :
      tab === "inventory" ? inventoryRecords.filter(r=>r.date>=dateFrom&&r.da
      tab === "payroll" ? payrollRecords.filter(r=>r.date>=dateFrom&&r.date<=
      overheadRecords.filter(r=>r.date>=dateFrom&&r.date<=dateTo),
      tab, `JSCAG-${tab}-${dateFrom}-to-${dateTo}`
    )>📄 I-Export lahat</button>
    </div>

    {tab === "costing" && <div className="card" style={{ padding: 0, overflow:
      <table>
        <thead><tr><th>Date</th><th>Ingredient</th><th>Payroll</th><th>Overhead
        <tbody>
          {costingRows.map(r => <tr key={r.date}>
            <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 600 }}>
            <td style={{ fontFamily: "'DM Mono',monospace" }}>{peso(r["Ingredie
            <td style={{ fontFamily: "'DM Mono',monospace" }}>{peso(r["Payroll"
            <td style={{ fontFamily: "'DM Mono',monospace" }}>{peso(r["Overhead
            <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700, co
            <td style={{ fontFamily: "'DM Mono',monospace", color: "#34d399" }}
            <td style={{ fontFamily: "'DM Mono',monospace", color: "#f87171" }}
            <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700, co
          </tr>)}
        </tbody>
      </table>
    </div>

    {tab === "inventory" && <div className="card" style={{ padding: 0, overflow
      <table>
        <thead><tr><th>Date</th><th>Item</th><th>Beginning</th><th>Ending</th><
        <tbody>
          {inventoryRecords.filter(r=>r.date>=dateFrom&&r.date<=dateTo).map(r =

```

```

const c = computeRecord(r, items), item = items.find(i=>i.id===r.it
return <tr key={r.id}>
  <td style={{ fontFamily: "'DM Mono',monospace" }}>{r.date}</td>
  <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{item?.name}</t
  <td style={{ fontFamily: "'DM Mono',monospace" }}>{r.beginning}</
  <td style={{ fontFamily: "'DM Mono',monospace" }}>{r.ending}</td>
  <td style={{ fontFamily: "'DM Mono',monospace", color: "#34d399",
  <td style={{ color: "#6b7390" }}>{r.unit}</td>
  <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700,
</tr>;
  }}}
</tbody>
</table>
</div>}

```

```

{tab === "payroll" && <div className="card" style={{ padding: 0, overflow:
  <table>
    <thead><tr><th>Date</th><th>Employee</th><th>Role</th><th>Daily Rate</t
    <tbody>
      {payrollRecords.filter(r=>r.date>=dateFrom&&r.date<=dateTo).map(r =>
        const c = computePayrollRow(r);
        return <tr key={r.id}>
          <td style={{ fontFamily: "'DM Mono',monospace" }}>{r.date}</td>
          <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{r.employee}</td>
          <td style={{ color: "#6b7390" }}>{r.role}</td>
          <td style={{ fontFamily: "'DM Mono',monospace" }}>{peso(r.dailyRa
          <td style={{ fontFamily: "'DM Mono',monospace" }}>{peso(r.allowan
          <td style={{ fontFamily: "'DM Mono',monospace", color: "#6b7390"
          <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700,
          <td><span className={`badge ${r.present ? "badge-green" : "badge-
        </tr>;
      }}}
    </tbody>
  </table>
</div>}

```

```

{tab === "overhead" && <div className="card" style={{ padding: 0, overflow:
  <table>
    <thead><tr><th>Date</th><th>Cost Type</th><th>Amount</th><th>Allocation
    <tbody>
      {overheadRecords.filter(r=>r.date>=dateFrom&&r.date<=dateTo).map(r =>
        <td style={{ fontFamily: "'DM Mono',monospace" }}>{r.date}</td>
        <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{r.costType}</td>
        <td style={{ fontFamily: "'DM Mono',monospace", fontWeight: 700, co
        <td><span className="badge badge-yellow">{r.allocationType}</span><
        <td style={{ color: "#6b7390" }}>{r.notes || "-"}</td>
      </tr>)}
    </tbody>
  </table>
</div>}

```

```

        </tbody>
      </table>
    </div>
  </div>
);
}

// — SETTINGS —————
function Settings() {
  const { settings, setSettings, categories, setCategories, products, setProducts } = useSettings();
  const [tab, setTab] = useState("general");
  const [form, setForm] = useState({ ...settings });
  const [newCat, setNewCat] = useState({ name: "", color: "#2563eb" });
  const [newProd, setNewProd] = useState({ name: "", sku: "", targetPackSize: "50" });

  function saveSettings() {
    setSettings(form);
    showToast("Settings na-save.");
  }

  const TABS = [
    { id: "general", label: "⚙️ General" },
    { id: "categories", label: "📁 Categories" },
    { id: "products", label: "📦 Products" },
    { id: "formulas", label: "📊 Formulas" },
    { id: "integrations", label: "🔗 Integrations" },
  ];

  return (
    <div>
      <div className="section-title">Settings</div>
      <div className="section-sub">I-customize ang sistema ayon sa inyong busines</div>

      <div style={{ display: "flex", gap: 8, marginBottom: 20, flexWrap: "wrap" }}>
        {TABS.map(t => <button key={t.id} className={`tab-btn ${tab === t.id ? "active" : ""}`}>{t.label}</button>)}
      </div>

      {tab === "general" && <div className="card" style={{ maxWidth: 560 }}>
        <div style={{ fontWeight: 700, fontSize: 15, marginBottom: 18 }}>General</div>
        <div style={{ display: "flex", flexDirection: "column", gap: 14 }}>
          <div><label>Company Name</label><input className="inp" value={form.comp}</div>
          <div><label>Tagline / Description</label><input className="inp" value={form.tagline}</div>
          <div><label>Currency Symbol</label><input className="inp" value={form.currency}</div>
          <div><label>Working Days per Month (para sa monthly-to-daily conversion)</label><input type="text" value={form.workingDays}</div>
          <div><label>Alert Email (para sa low stock notifications)</label><input type="text" value={form.alertEmail}</div>
          <div><label>Reject Rate Threshold (%) – Mag-aalert kapag naabot na</label><input type="text" value={form.rejectRate}</div>
          <div style={{ display: "flex", alignItems: "center", gap: 10 }}>
            <button type="button" className="btn btn-sm">Cancel</button>
            <button type="button" className="btn btn-sm">Save</button>
          </div>
        </div>
      </div>
    </div>
  );
}

```

```

        <input type="checkbox" checked={form.lowStockAlerts} onChange={e => s
        <span style={{ fontSize: 13 }}>I-enable ang Low Stock Alerts</span>
    </div>
    <button className="btn btn-primary" onClick={saveSettings}><img alt="Save icon" data-bbox="775 103 798 118"/> I-Save an
  </div>
</div>}

```

```

{tab === "categories" && <div>
  <div className="card" style={{ marginBottom: 16 }}>
    <div style={{ fontWeight: 700, fontSize: 15, marginBottom: 14 }}>Mag-ad
    <div style={{ display: "flex", gap: 10, flexWrap: "wrap" }}>
      <div style={{ flex: 1, minWidth: 180 }}><label>Category Name</label><
      <div><label>Color</label><input type="color" value={newCat.color} onC
      <div style={{ padding: 20 }}><button className="btn btn-primary" o
        if (!newCat.name.trim()) return;
        setCategories([...categories, {...newCat, id: Date.now(), active: t
        setNewCat({ name: "", color: "#2563eb" }); showToast("Category na-a
      })>+ Add</button></div>
    </div>
  </div>
</div>
<div className="card" style={{ padding: 0, overflow: "hidden" }}>
  <table>
    <thead><tr><th>Category</th><th>Color</th><th>Status</th><th></th></tr>
    <tbody>
      {categories.map(c => <tr key={c.id}>
        <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{c.name}</td>
        <td><div style={{ width: 24, height: 24, borderRadius: 6, backgro
        <td><span className={`badge ${c.active ? "badge-green" : "badge-r
        <td><button className="btn btn-ghost btn-sm" onClick={() => { set
      </tr>)}
    </tbody>
  </table>
</div>
</div>}

```

```

{tab === "products" && <div>
  <div className="card" style={{ marginBottom: 16 }}>
    <div style={{ fontWeight: 700, fontSize: 15, marginBottom: 14 }}>Mag-ad
    <div className="grid-2" style={{ gap: 12 }}>
      <div><label>Product Name</label><input className="inp" value={newProd
      <div><label>SKU</label><input className="inp" value={newProd.sku} onC
      <div><label>Target Pack Size</label><input className="inp" value={new
      <div><label>Selling Price (₱)</label><input className="inp" type="num
    </div>
    <button className="btn btn-primary" style={{ marginTop: 14 }} onClick={
      if (!newProd.name.trim()) return;
      setProducts([...products, {...newProd, id: Date.now()}]);
    }
  </div>

```

```

      setNewProd({ name: "", sku: "", targetPackSize: "50g", sellingPrice:
    })>+ Add Product</button>
  </div>
  <div className="card" style={{ padding: 0, overflow: "hidden" }}>
    <table>
      <thead><tr><th>Product</th><th>SKU</th><th>Pack Size</th><th>Selling
    <tbody>
      {products.map(p => <tr key={p.id}>
        <td style={{ fontWeight: 600, color: "#e8eaf0" }}>{p.name}</td>
        <td style={{ fontFamily: "'DM Mono',monospace", color: "#6b7390"
        <td>{p.targetPackSize}</td>
        <td style={{ fontFamily: "'DM Mono',monospace", color: "#34d399"
        <td><span className={`badge ${p.active ? "badge-green" : "badge-r
      </tr>)}
    </tbody>
  </table>
</div>
</div>}

```

```

{tab === "formulas" && <div className="card" style={{ maxWidth: 620 }}>
  <div style={{ fontWeight: 700, fontSize: 15, marginBottom: 16 }}>1234 Defau
  <div style={{ fontSize: 13, color: "#8b92a8", marginBottom: 20 }}>Ito ang
  [
    { label: "Used Quantity", formula: "Beginning Inventory - Ending Invent
    { label: "Used Cost (per item)", formula: "Used Quantity × Cost Per Uni
    { label: "Cost Per Unit (g)", formula: "Purchase Price ÷ 1000 (para sa
    { label: "Cost Per Unit (ml)", formula: "Purchase Price ÷ 1000 (para sa
    { label: "Daily Payroll Cost", formula: "Σ (Daily Rate + Allowance + SS
    { label: "Monthly → Daily Overhead", formula: "Monthly Amount ÷ Working
    { label: "Total Daily Production Cost", formula: "Total Ingredient Cost
    { label: "Cost Per Pack 🌟", formula: "Total Daily Production Cost ÷ Tc
    { label: "Gross Margin", formula: "Selling Price - Cost Per Pack" },
    { label: "Gross Margin %", formula: "(Gross Margin ÷ Selling Price) × 1
  ].map(f => <div key={f.label} style={{ marginBottom: 14, padding: "12px 1
    <div style={{ fontSize: 11, color: "#6b7390", fontWeight: 700, marginBo
    <div style={{ fontFamily: "'DM Mono',monospace", fontSize: 13, color: "
  </div>)}
</div>}

```

```

{tab === "integrations" && <div className="card" style={{ maxWidth: 560 }}>
  <div style={{ fontWeight: 700, fontSize: 15, marginBottom: 16 }}>🔗 Integ
  [
    { name: "Supabase (Database)", status: "Kailangan i-connect", desc: "I-
    { name: "Resend (Email Alerts)", status: "Kailangan i-setup", desc: "I-
    { name: "GitHub + Vercel (Deployment)", status: "Kailangan i-connect",
    { name: "SMS Alerts (Semaphore PH)", status: "Future version", desc: "A
    { name: "Excel Export (xlsx)", status: "Demo mode (CSV)", desc: "Sa pro

```

```

].map(i => <div key={i.name} style={{ marginBottom: 12, padding: "14px 16
  <div style={{ display: "flex", justifyContent: "space-between", alignIt
    <span style={{ fontWeight: 700, color: "#e8eaf0", fontSize: 13 }}>{i.
    <span className="badge" style={{ background: i.color+"22", color: i.c
  </div>
  <div style={{ fontSize: 12, color: "#6b7390" }}>{i.desc}</div>
</div>}}
</div>}
</div>
);
}

// — SHARED MODAL —————
function FormModal({ title, children, onClose, onSave }) {
  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal" onClick={e => e.stopPropagation()}>
        <div style={{ fontWeight: 700, fontSize: 16, marginBottom: 20, color: "#e
          {children}
        <div style={{ display: "flex", gap: 10, marginTop: 22 }}>
          <button className="btn btn-primary" onClick={onSave}>&img alt="floppy disk icon" data-bbox="718 428 741 444"/> I-Save</button>
          <button className="btn btn-ghost" onClick={onClose}>Kanselahin</button>
        </div>
      </div>
    </div>
  );
}

```